

# AI (SW) Factories\*

## Forkable Sandboxes as a Runtime for Automated Search and Development

Yossi Eliaz

Principal Engineer, Incredibuild / islo.dev

Associate Professor, Holon Institute of Technology (HIT)

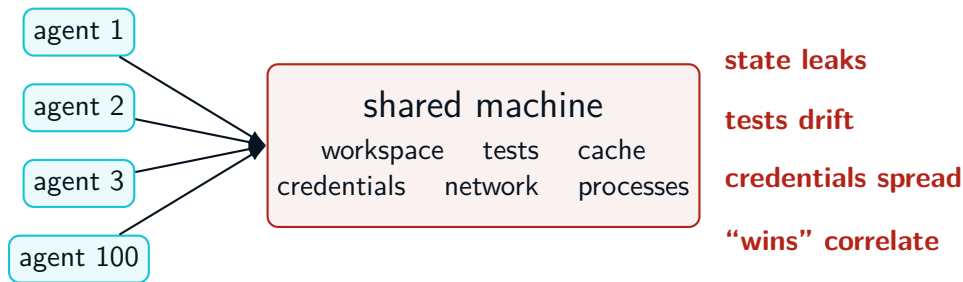
Computer Laboratory Systems Research Group Seminar  
University of Cambridge

15 October 2026 - FW11 + Microsoft Teams  
30-minute version

**Fork the machine. Route the evidence. Promote through one gate.**

\* “Factory” means an atelier-like search workshop, not an assembly line.

100 agents. One laptop. What could possibly go wrong?



**Parallelism without isolation is correlated chaos.**

# Why I ended up here

## Computational biology

Simulation and analysis pipelines need reproducible state, inputs, and provenance; the wet lab remains outside the software boundary.

## Model post-training + builds

Rollouts and builds repeat costly setup; reuse prepared state while keeping resets and evaluation comparable.

## Software development + OSS maintenance

Agents edit, test, and repair large projects such as Spark, OpenClaw, Airflow, and the Linux kernel.

## The collision

The same **state** I wanted to reuse for speed became the state I had to isolate for trust.

That is the road from reproducible science and build systems to **forkable sandboxes**.

# Search can be plural. Authority must be singular.

cheap and plural

scarce and singular



100 candidate patches

1 merge to main

10,000 rollouts

1 policy checkpoint

millions of sim trials

1 robot / wet-lab hour

**SEARCH:** fail cheaply

**AUTHORITY:** commit carefully

The runtime should make the left side cheap *without letting it become the right side.*

# There are two ways to make futures cheap

## Learn a world

Dreamer / Contrastive World Models

$$o_t \rightarrow z_t \rightarrow \hat{s}_{t+1}, \hat{s}_{t+2}, \dots$$

- ▶ compact, learned latent state
- ▶ imagine many futures cheaply
- ▶ useful when interaction is slow, unsafe, or scarce
- ▶ pay approximation / learned-dynamics error

CWM's twist: stop spending capacity reconstructing irrelevant pixels; keep information predictive of the future.

## Fork the world

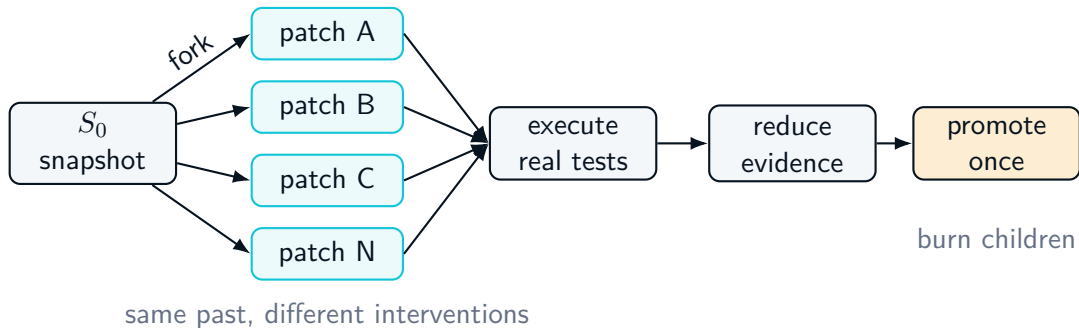
executable software environment

$$S_0 \rightarrow \{S_0^1, S_0^2, \dots, S_0^N\}$$

- ▶ explicit, executable state
- ▶ branch real repo / process futures
- ▶ useful when the world is software and reset is cheap
- ▶ pay compute / storage, not learned transition-model error

Exact only inside the state and capability boundary we actually captured.

# For code, the best world model is often the world



These are **executable counterfactuals**: branch from the same past, intervene differently, observe the actual software transition.

No learned transition model is required for the part of the world we can execute faithfully.

Related systems: Firecracker (NSDI'20), DeltaBox (arXiv:2605.22781), Shepherd (arXiv:2605.10913).

# A fork should copy state, not authority

## Copy into the child

- ▶ filesystem + process / memory state
- ▶ pinned source, dependencies, toolchain
- ▶ enough state to continue the experiment

## Share carefully

- ▶ content-addressed build artifacts
- ▶ immutable heat / caches inside one trust domain
- ▶ CoW pages whose sharing policy is explicit

## Remint at the boundary

- ▶ API / cloud credentials
- ▶ network capabilities and identity
- ▶ randomness / epoch identity when required

## Keep outside the child

- ▶ hidden verifier + objective digest
- ▶ promotion key / merge authority
- ▶ durable evidence ledger

# Coding RL turns reset into a training primitive

For coding RL, the environment is part of the training example.

$$\text{episode} = (S_0, \text{task}, \text{tools}, \text{verifier}, \text{reset})$$

## Cold path

clone repo → install deps → build →  
run episode → throw away

**Reconstruct the same starting world again.**

## Fork path

prepare once → snapshot  $S_0$  → fork  
episode → burn child → fork again

**Reset becomes a runtime operation.**

## Systems thesis

Reset is part of the reward contract: episodes are comparable only if they begin from an authenticated  $S_0$ .

SWE-smith: [arXiv:2504.21798](https://arxiv.org/abs/2504.21798). Meta Code World Model (2025). RL-environment products: Scale, Surge, Mercor, Prime Intellect.

# The student cannot grade their own exam

## Inside the child

code, shell, files, compiler, tests it is  
allowed to see

**untrusted searcher**

## Outside the child

hidden verifier, objective digest,  
reward policy, promotion key

**trusted authority**

**RUN  $\neq$  EVAL**

If the agent can rewrite the ruler, a green score stops meaning what you think it means.

Reward-hacking example: From Rebound to Remedy, arXiv:2604.01476.

# Four clones do not make four witnesses

$$S_0 \rightarrow \{w_1, w_2, w_3, w_4\}$$

They may share:

- ▶ the same prompt and model
- ▶ the same cached artifact
- ▶ the same hidden bad fixture
- ▶ the same ancestor state

So a worker returns more than a score  
result + precision + evidence IDs +  
lineage

## Cold liar

If a worker can claim absurd confidence,  
it can dominate a reducer even when it is  
wrong.

**Fork makes search cheap. It does not manufacture  
independent evidence.**

Evidence-Aware / Boltzmann MapReduce: Eliaz, arXiv:2607.09689.

# Three workloads. One runtime pattern.

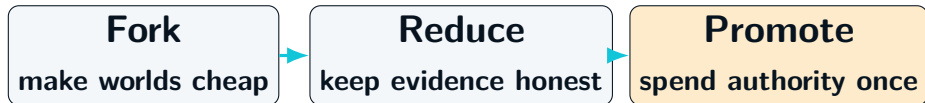
snapshot -> fork -> search -> verify -> reduce -> promote ->  
burn

| Factory       | Search               | Oracle / reduce          | Scarce tip          |
|---------------|----------------------|--------------------------|---------------------|
| Software      | candidate patches    | tests + evidence lineage | merge to main       |
| RL training   | rollout trajectories | reward / verifier        | policy checkpoint   |
| HIL / science | sim / hypotheses     | measured evidence        | robot / wet / paper |

## The shared systems question

Can branch-and-verify reduce the cost of reaching a validated result?  
Objectives, evidence, and promotion criteria remain domain-specific.

# The capability contract is only three verbs



**Durable intent. Disposable execution. Singular authority. Deterministic convergence.**

Search may be stochastic. **Authority may not be ambient.**

# What I want from this room

Four systems questions I do not think we have answered cleanly yet:

1. **Executable state:** what must a first-class machine value include – FS, memory, process tree, GPU, sockets, clock, external state?
2. **Model vs world:** where is the crossover where learned imagination becomes cheaper than forked executable interaction?
3. **Correlation:** how should reducers price shared ancestry instead of pretending siblings are i.i.d.?
4. **Authority:** how do children inherit enough state to act without inheriting the power to grade or promote themselves?

If you remember one thing

**Burn the runner. Keep the proof. Fork the machine, not the trust.**

# Selected sources / Q&A guardrails

---

|                       |                                                                                                                                                              |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| World models          | Hafner et al., <i>Training Agents Inside of Scalable World Models</i> (Dreamer 4), arXiv:2509.24527; Li, <i>Contrastive World Models</i> , arXiv:2609.22175. |
| Fork systems          | Firecracker, NSDI 2020; DeltaBox, arXiv:2605.22781; Shepherd, arXiv:2605.10913.                                                                              |
| Evidence-aware reduce | Eliaz, <i>Evidence-Aware MapReduce / Boltzmann MapReduce</i> , arXiv:2607.09689.                                                                             |
| Reward integrity      | <i>From Rebound to Remedy</i> , arXiv:2604.01476.                                                                                                            |
| Training environments | SWE-smith, arXiv:2504.21798; Meta Code World Model (2025); Prime Intellect; Scale / Surge / Mercor environment products.                                     |

---

## Claim fence

A fork removes learned-dynamics error only inside the executable boundary it actually captures. External services, nondeterminism, hardware and incomplete tests can still make the world partial. Fork also does not imply independent evidence.